

The `/process-async` Request Lifecycle & Document Notification Pipeline

A complete, code-grounded walkthrough of `wait=true` vs `wait=false`, the downstream extraction pipeline, and both notification channels — webhooks and WebSockets.



`kdext_api`



`kdext_document_api_v2`

Endpoint

`POST /v1/documents/process-async`

Notification channels

`webhooks.py` + `events_ws.py` (+ `status_ws.py`)

Prepared

7 August 2026

Sourced from

direct code reading, file:line cited throughout

CONTENTS

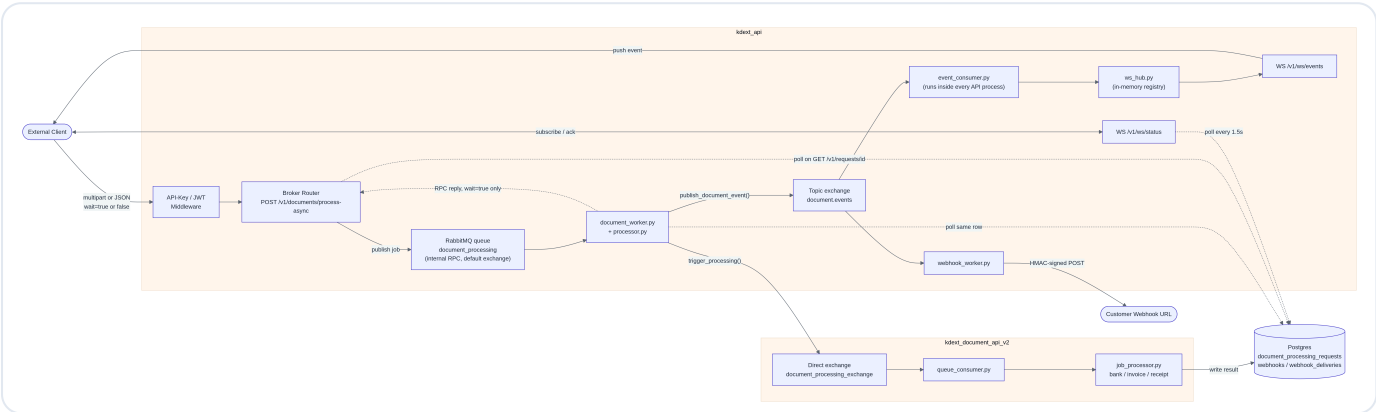
1. System overview & architecture How the two repos, three queues, two databases and two notification channels fit together
 2. The request contract Every way `wait` can be supplied, and how precedence is resolved
 3. Flow A — `wait=false` (fire-and-forget) Step-by-step, with sequence diagram
 4. Flow B — `wait=true` (blocking RPC) The AMQP direct-reply pattern, with sequence diagram
 5. Complete response catalog Every HTTP status this endpoint (and its middleware) can return
 6. Downstream processing in `kdext_document_api_v2` Queue topology, job routing, retries, dead-lettering
 7. Notifications — Webhooks `webhooks.py` : registration, signing, delivery, retry/backoff
 8. Notifications — WebSocket `/v1/ws/events` `events_ws.py` + `ws_hub.py` + `event_consumer.py`
 9. Companion WebSocket `/v1/ws/status` `status_ws.py` : per-request subscriptions and DB polling
 10. End-to-end timeline How the HTTP response and the two notification channels overlap in time
 11. Known behavioral quirks Stale docstrings, swallowed status codes, enum drift, DLQ gaps
 12. Appendix Config/timeout reference, status enum reconciliation, full endpoint index
-

SECTION 1

System Overview & Architecture

A single call to `POST /v1/documents/process-async` touches two independent services, three RabbitMQ topologies, a shared Postgres database, and — once the document finishes — two parallel notification channels. This section is the map; every later section zooms into one box.

FIGURE 1 — COMPONENT & DATA-FLOW OVERVIEW



Solid arrows = synchronous call or message publish. Dashed arrows = polling / async reply. The two services never call each other directly — they only ever meet at the shared `document_processing_requests` Postgres row.

The three queues, in one line each

BROKER / QUEUE	OWNER	PURPOSE
<code>document_processing</code> (default exchange, RPC)	<code>kdext_api</code>	Internal hand-off from the HTTP router (<code>broker.py</code>) to <code>kdext_api</code> 's own worker (<code>document_worker.py</code>). Carries the raw upload metadata. This is the queue that <code>wait=true</code> 's reply-queue mechanics attach to.
<code>document_processing_exchange</code> / <code>document_processing_queue</code> (direct exchange)	<code>kdext_document_api_v2</code>	The actual extraction hand-off. Carries only <code>{"request_id", "published_at"}</code> — the heavy payload already lives in Postgres.
<code>document.events</code> (topic exchange)	<code>kdext_api</code>	Fan-out for the three customer-facing lifecycle events. Two independent consumers bind to it: <code>webhook_worker.py</code> and, inside every API process, <code>event_consumer.py</code> .

KEY ARCHITECTURAL FACT

`kdext_document_api_v2` never calls `kdext_api` back — not over HTTP, not over RabbitMQ. It writes its result straight into the shared `document_processing_requests` Postgres row. `kdext_api`'s worker (and the `GET /v1/requests/{id}` route, and the `wait=true` RPC reply) all learn the outcome by reading that same row. See Section 6.

The Request Contract

`app/routers/broker.py:156` — `@router.post("/process-async")`, mounted under prefix `/v1/documents`.

Body shapes (mutually exclusive by Content-Type)

CONTENT-TYPE	FIELDS
<code>multipart/form-data</code>	<code>file</code> (required <code>UploadFile</code>) · <code>document_type</code> (optional hint) · <code>clientDocumentType</code> (optional, form alias for <code>client_document_type</code>) · <code>wait</code> (form field, optional)
<code>application/json</code>	<code>s3_url</code> / <code>url</code> / <code>public_url</code> (one required, must start with <code>http://</code> or <code>https://</code>) · <code>document_type</code> · <code>clientDocumentType</code> · <code>wait</code> · <code>file_size</code>

How `wait` is actually resolved

`wait` is declared as a plain `str` on both the form field and the `?wait=` query param — FastAPI does *not* coerce it to `bool`. A private helper does the string-to-bool parsing:

```
# broker.py:64-72
def _parse_wait_flag(value) -> bool:
    if value is None:
        return False
    if isinstance(value, bool):
        return value
    return str(value).strip().lower() in ("1", "true", "yes", "on")
```

Precedence: query string > JSON body > multipart form field. If none are supplied, the default is **False** — a bare `POST` with no `wait` anywhere is always fire-and-forget.

SOURCE	WINS WHEN...	ACCEPTED TRUTHY VALUES
<code>?wait=</code> (query)	Present at all — overrides both body and form.	<code>1</code> , <code>true</code> , <code>yes</code> , <code>on</code> (case-insensitive, trimmed)
JSON body <code>"wait"</code>	Content-Type is JSON, query param absent.	same set
Multipart form field <code>wait</code>	Neither of the above supplied a value.	same set

PRACTICAL IMPLICATION

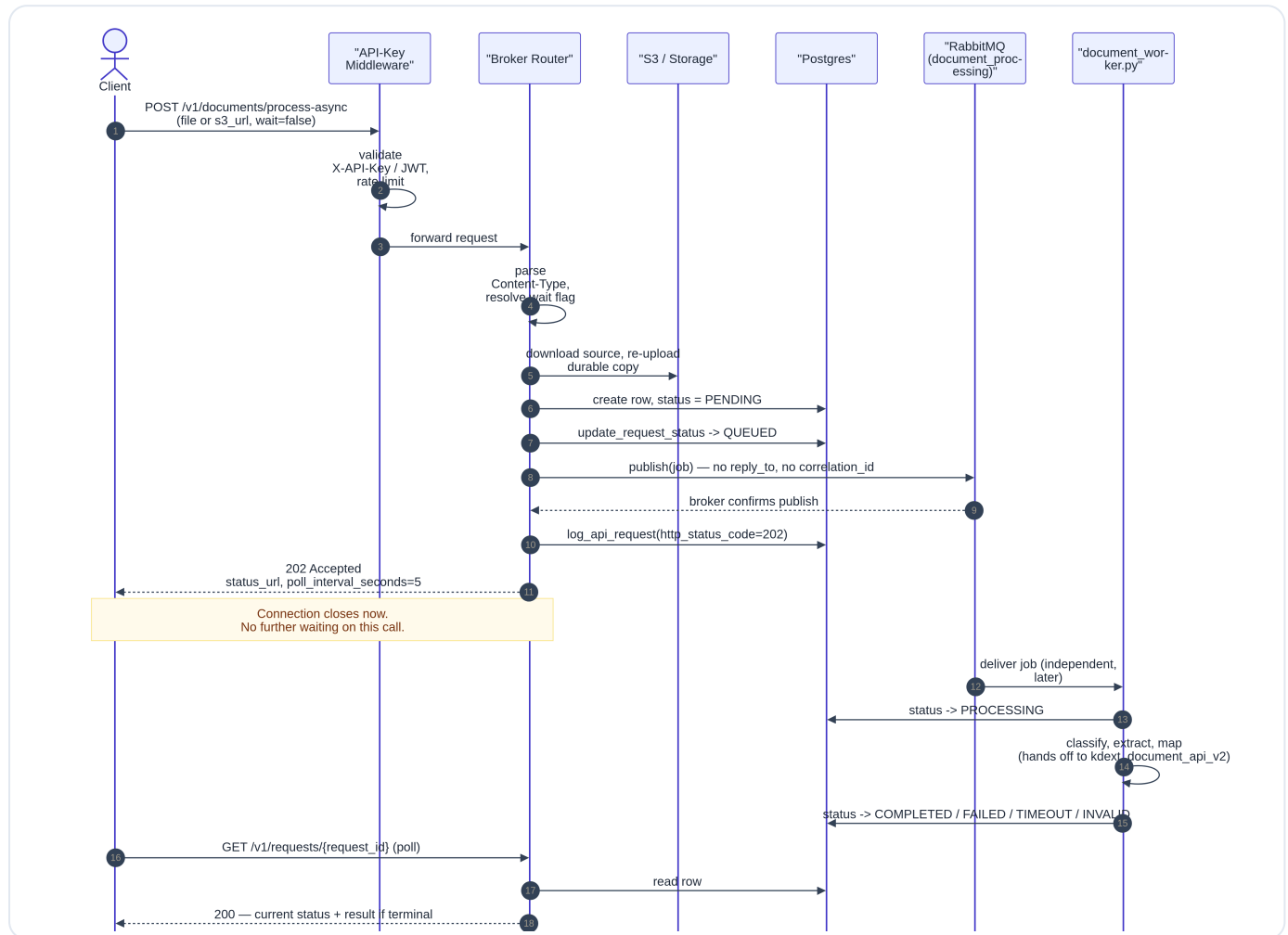
Because `wait` is string-typed everywhere, `?wait=false` in the query string is **still fire-and-forget** — but so is `?wait=banana`, `?wait=0`, or simply omitting it. Only the exact tokens above flip it to blocking mode. Clients that pass a JSON boolean `true` also work, since `_parse_wait_flag` special-cases real `bool` input.

SECTION 3

Flow A — `wait=false` (fire-and-forget, the default)

The HTTP response is sent the instant the job message is confirmed published (or confirmed *failed* to publish) — never after the worker actually processes anything.

FIGURE 2 — WAIT=FALSE SEQUENCE



Step by step

- `request_id = "req_" + uuid4().hex` is minted; `start_time` recorded. *broker.py:238-239*
- Content-Type is sniffed — must contain `application/json` or `multipart/form-data`, else immediate `400`. *broker.py:248-255*
- `should_wait` resolved per Section 2 — `False` here. *broker.py:246*
- Input resolved: JSON path downloads from `s3_url` via `asyncio.to_thread(download_s3_to_path, ...)`; multipart path streams `file` to a temp file via `asyncio.to_thread(shutil.copyfileobj, ...)`. *broker.py:264-320*
- Size guard against the temp file — `> MAX_FILE_SIZE` (500 MB default) returns `413`. *broker.py:327-337*
- The file is **re-uploaded to S3 as `kdex_api`'s own durable copy** via `DocumentService.save_uploaded_file` — done *before* classification so a crash never loses the

original. *broker.py:339-367*

7. Classification is **deliberately skipped here** — `confidence` and `document_type` stay `None` on the response; classification now happens later, off the request's critical path, inside the worker. *broker.py:369-384*
8. DB row created with `status="PENDING"`, then immediately updated to `"QUEUED"`. *broker.py:396-400, 126-153*
9. Job dict built: `request_id, user_id, api_key, s3_url, filename, file_size, client_document_type, pre_classified: false, ip_address, user_agent, key_environment`. *broker.py:426-457*
10. **Fire-and-forget publish** — `rpc_client.publish(job)` builds a persistent AMQP message with **no `reply_to`**, no `correlation_id`, routed by name to the `document_processing` queue on the default exchange. *broker.py:482; rabbitmq.py:246-262*
11. On publish success → `202 Accepted` tracking payload (Response S1, Section 5). On publish failure after retries → DB forced to `FAILED`, `503` returned (Response F1, Section 5). *broker.py:478-537*
12. `finally`: local temp file removed. *broker.py:609-614*

WHAT THE CLIENT IS EXPECTED TO DO NEXT

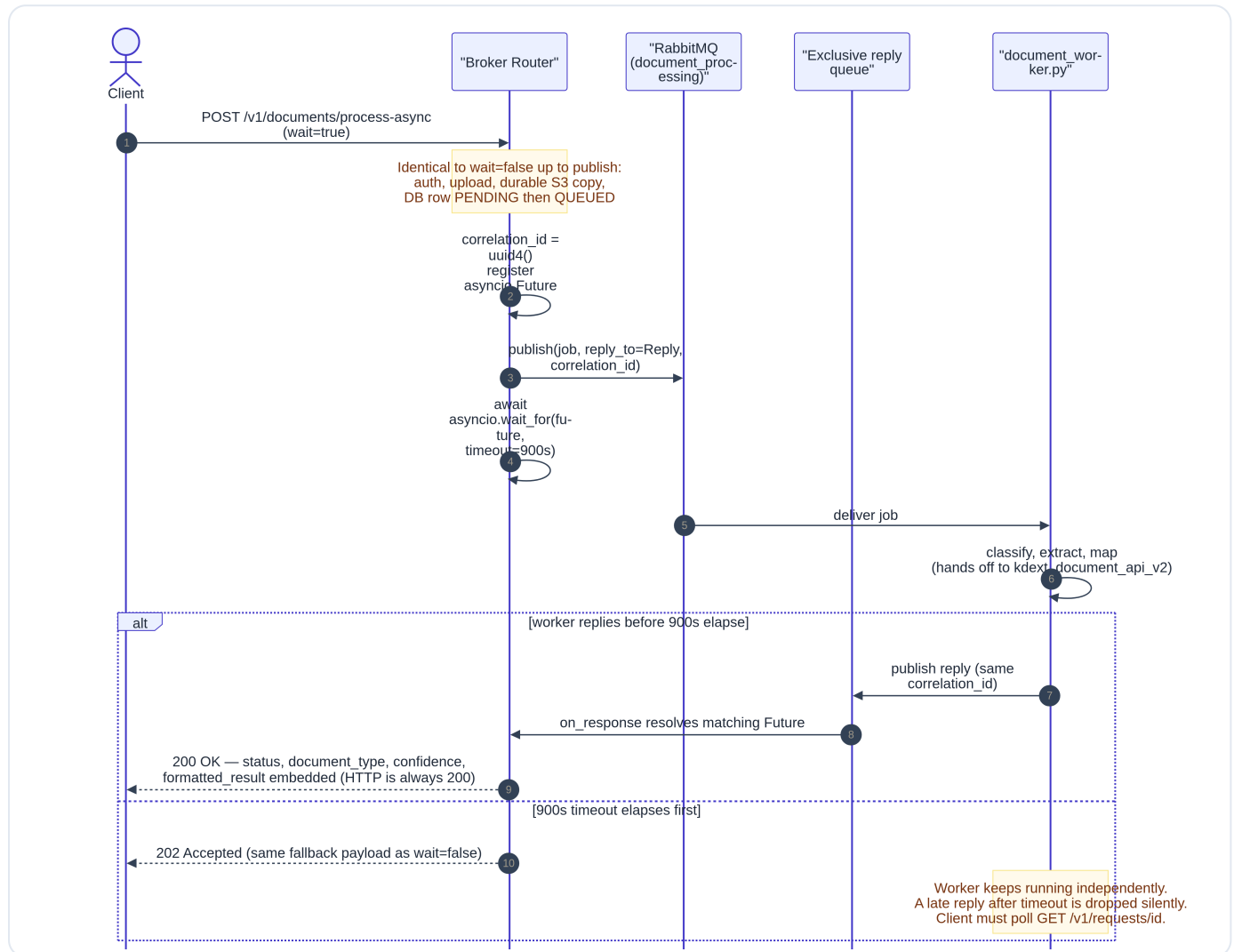
Poll `GET /v1/requests/{request_id}` until `status` is `completed` / `failed` /etc, or simply wait for a webhook / WebSocket push (Sections 7–9). The 202 body embeds `poll_interval_seconds: 5` and a matching `Retry-After: 5` header as a hint.

SECTION 4

Flow B — `wait=true` (blocking RPC)

Steps 1–9 from Flow A are byte-for-byte identical. The divergence is entirely in how the message is published and what happens after.

FIGURE 3 — WAIT=TRUE SEQUENCE



The mechanism: classic AMQP direct-reply RPC — not polling, not a shared

`asyncio.Event`

1. A fresh `correlation_id = uuid4()` is minted; an `asyncio.Future` is registered in the RPC client's in-memory `self._futures[correlation_id]` map. *rabbitmq.py:208-244*
2. The job is published with `correlation_id` set and `reply_to` pointed at this API process's own **exclusive, auto-delete callback queue** (one per connection, declared once in `connect()`). *rabbitmq.py:69-72, 238*
3. The router then does exactly this: `return await asyncio.wait_for(future, timeout=settings.RABBITMQ_RPC_TIMEOUT)` — timeout defaults to **900 seconds**. *broker.py:541-543; config.py:222-225*

4. The worker, once done, publishes its reply with the **same** `correlation_id` to the `reply_to` queue name. *document_worker.py:56-69*
5. The client's callback-queue consumer (`_on_response`) pops the matching future by `correlation_id` and resolves it with the decoded JSON payload. A late reply after timeout finds no matching (or already-resolved) future and is silently dropped — the DB row it caused is unaffected. *rabbitmq.py:98-116*

On success — HTTP 200, but `status` inside the body may not mean success

```
# broker.py:564-596 – always status_code=200, regardless of result["status"]
final_result = {
    "request_id": request_id, "status": result.get("status"),
    "validation": result.get("validation"), "document_type": result.get("document_type"),
    "client_document_type": result.get("client_document_type"),
    "confidence": result.get("confidence"), "message": result.get("message"),
    "document_url": s3_url, "formatted_result": result.get("formatted_result"),
}
return Response(content=toon.encode(final_result), media_type="text/plain", status_code=200)
```

GOTCHA

HTTP 200 does not mean the document processed successfully. If the worker's RPC reply carries `"status": "failed"` or `"status": "invalid"`, the wire-level HTTP status is *still 200* — only the embedded `status` string tells the real story. Callers using `wait=true` must inspect the body, not just the status code.

On timeout — falls back to the exact same 202 payload as fire-and-forget

After 900 seconds with no reply, the router does **not** return 504/408. It returns the identical `_accepted_response(...)` tracking payload used by Flow A's success case (Response S1). The worker keeps running independently; the client is expected to fall back to polling `GET /v1/requests/{request_id}`. *broker.py:544-555*

SECTION 5

Complete Response Catalog

Every distinct HTTP response reachable from a call to `/v1/documents/process-async`, from the outermost middleware down to the router's own catch-all. Router-level responses use `Response(media_type="text/plain")` with a **TOON-encoded** body (shown below as the underlying dict) — **not** standard `application/json`. Middleware/dependency-layer responses *do* use real `JSONResponse`.

5.1 — Middleware & auth layer (fires before the router body ever runs)

CODE	LAYER	CONDITION	BODY
401	API-key middleware	No <code>X-API-Key</code> header present	<code>{error, message, request_id}</code> <i>api_key_middleware.py:99-108</i>
403	API-key middleware	Key doesn't resolve to any user	<code>{error: "Invalid API key", ...}</code> <i>api_key_middleware.py:126-135</i>
403	API-key middleware	Key valid but <code>keys_active=false</code> (pending admin approval)	<code>{error: "API key pending approval", ...}</code> <i>api_key_middleware.py:139-153</i>
429	API-key middleware	Rate limit exceeded (default 120/min, fails open if Redis is down)	<code>{error, message, request_id}</code> + <code>Retry-After</code> , <code>X-RateLimit-*</code> headers <i>api_key_middleware.py:164-187</i>
401	JWT dependency (no API key used)	No bearer token and no middleware-set <code>user_id</code>	<code>{detail: "Not authenticated..."}</code> <i>dependencies.py:148-152</i>
401	JWT dependency	Invalid / expired JWT	<code>{detail: "Could not validate credentials"}</code> <i>dependencies.py:72-88</i>
403	JWT dependency	Token decodes to an internal (non-external) user	<code>{detail: "Not authorized. External user access only."}</code> <i>dependencies.py:158-162</i>
403	JWT dependency	User account inactive	<code>{detail: "Inactive user"}</code> <i>dependencies.py:99-103</i>
404	JWT dependency	Middleware set <code>user_id</code> but the row is gone from Postgres	<code>{detail: "User not found"}</code> <i>dependencies.py:139-143</i>

5.2 — Router body (`broker.py`) — validation branches

CODE	ID	CONDITION	BODY
400	B1	Content-Type is neither JSON nor multipart	<code>{error: "Content-Type must be application/json or multipart/form-data"}</code> <i>broker.py:248-255</i>

400	B2	JSON Content-Type but body fails to parse	<code>{error: "Invalid JSON body"}</code> <i>broker.py:264-272</i>
400	B3	JSON body missing <code>s3_url / url / public_url</code>	<code>{error: "s3_url (or url) is required"}</code> <i>broker.py:279-289</i>
400	B4	Supplied URL doesn't start with <code>http:// / https://</code>	<code>{error: "url must be an http(s) URL"}</code> <i>broker.py:290-295</i>
400	B5	Multipart request with no <code>file</code> / empty filename	<code>{error: "file is required"}</code> <i>broker.py:305-310</i>
413	B6	Uploaded file exceeds <code>MAX_FILE_SIZE</code> (500 MB)	<code>{request_id, status: "failed", error: "File too large. Maximum size: 500.0MB"}</code> <i>broker.py:327-337</i>

5.3 — Router body — dispatch outcomes

CODE	ID	MODE	CONDITION	BODY
202	S1	both	Publish confirmed (wait=false), or the RPC reply never arrived within 900s (wait=true fallback)	<code>{request_id, status: "processing", status_url, document_url, filename, document_type: null, confidence: null, submitted_at, poll_interval_seconds: 5, retry_after: 5, result_retained: true, message}</code> + <code>Retry-After: 5</code> header <i>broker.py:75-110, 534-555</i>
200	S2	wait=true	RPC reply received before timeout — status_code is always 200 even if the embedded status is failed / invalid	<code>{request_id, status, validation, document_type, client_document_type, confidence, message, document_url, formatted_result}</code> <i>broker.py:564-596</i>
503	F1	wait=false	Publish to RabbitMQ failed after <code>RABBITMQ_PUBLISH_MAX_RETRIES</code> (3) attempts — DB row force-marked <code>FAILED</code>	<code>{request_id, status: "failed", error: "Could not queue the document for processing. Please retry."}</code> + <code>Retry-After: 5</code> <i>broker.py:478-516</i>
500	E1	both	Any uncaught exception anywhere in the handler body (the catch-all) — including HTTPExceptions raised deeper in the call stack, which lose their original status code here	<code>{request_id, error: client_safe_error(exc)}</code> <i>broker.py:598-608</i>

GOTCHA — SWALLOWED STATUS CODES

`DocumentService.save_uploaded_file` can raise `HTTPException(400)` (disallowed extension) or `HTTPException(413)` (local-storage overflow). Because `broker.py` has no `except HTTPException:` `raise` guard around that call, both are caught by the generic `except Exception` at the bottom and re-emitted as a flat **500**, with the original code/message buried inside `client_safe_error(exc)`'s stringified text.

5.4 — Response schema fields (`GET /v1/requests/{request_id}` and the `wait=true` body)

FIELD	TYPE	NOTES
<code>request_id</code>	str	<code>req_<uuid4 hex></code>
<code>status</code>	str	lowercased DB value — see Section 11 enum reconciliation
<code>filename</code>	str	
<code>submitted_at</code> / <code>completed_at</code>	datetime / null	
<code>processing_time_ms</code>	int / null	
<code>result</code> / <code>formatted_result</code>	object / null	raw pipeline output vs. mapped/UI-shaped output
<code>validation</code>	bool / null	
<code>error_message</code>	str / null	
<code>document_type</code> / <code>client_document_type</code>	str / null	

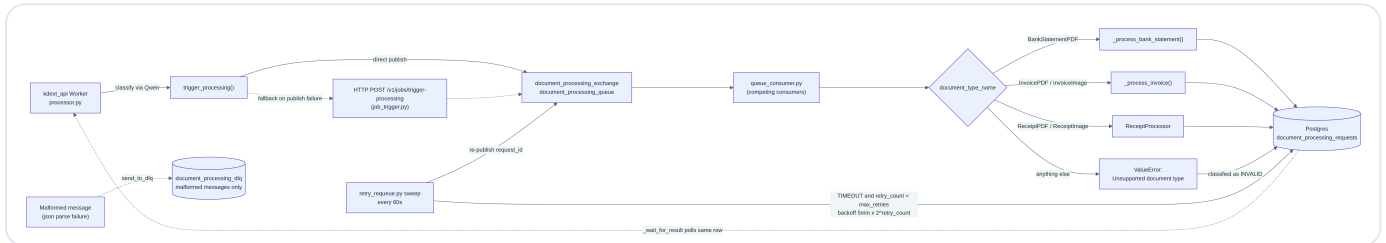
Note: `broker.py`'s route has no `response_model` at all; `GET /v1/requests/{id}` declares `response_model=DocumentProcessingResult` but then hand-builds and returns its own dict directly, so the declared model is cosmetic (OpenAPI docs only) and is never actually validated against the wire response. *requests.py:457, 534-556*

SECTION 6

Downstream Processing — kdxext_document_api_v2

Once kdxext_api's own worker classifies the document, it hands the job to kdxext_document_api_v2 for extraction. This is a second, entirely separate queue hop.

FIGURE 4 — HAND-OFF, ROUTING, RETRY SWEEP AND DEAD-LETTERING



The hand-off

`DocumentService.trigger_processing()` (inside `kdxext_api`'s worker) publishes a minimal `{"request_id", "published_at"}` message directly onto `document_processing_exchange` / `document_processing_queue`. If that direct publish throws, it falls back to an HTTP POST against `kdxext_document_api_v2`'s own `/v1/jobs/trigger-processing` (`job_trigger.py`), which performs the identical publish on the receiving side. Either path lands the same message on the same queue.

document_service.py:480-541; job_trigger.py:176

Pipeline stages inside kdxext_document_api_v2

1. `queue_consumer.py` holds a durable, competing-consumer subscription (QoS/prefetch = `MAX_CONCURRENT_JOBS`, default 5); each delivery spawns its own `asyncio.Task`.
queue_consumer.py:119-133, 290-294
2. Message body is JSON-decoded for `request_id`. A parse failure is the **only** code path that sends to the dead-letter queue, then acks. *queue_consumer.py:296-309*
3. `AsyncJobProcessor.process_job(request_id)` re-fetches the DB row, bails if already terminal, flips status to `PROCESSING`. *job_processor.py:341-392*
4. File resolved from S3 (or a shared local path fallback). *job_processor.py:396-455*
5. **Routed by** `document_type_name` (already assigned upstream): `BankStatementPDF` → bank-statement processor · `InvoicePDF` / `InvoiceImage` → invoice processor + mapper · `ReceiptPDF` / `ReceiptImage` → receipt processor + mapper. Anything else raises `ValueError("Unsupported document type")`, which becomes `INVALID`. *job_processor.py:488-628*
6. A stale-claim check re-reads the row before extraction, before the success write, and before the failure write — guards against a slower attempt clobbering a newer retry's result. *job_processor.py:481, 687, 760, 1516-1533*
7. **Result is written straight to Postgres** — `complete_processing_request()` / `fail_processing_request()` / `timeout_processing_request()` / `invalidate_processing_request()`. No message is ever published back, and no HTTP callback is ever made to `kdxext_api`. *database_manager.py:246-425*

8. The message is **always ACKed** after step 7, regardless of outcome — the queue consumer never re-decides retry eligibility itself. *queue_consumer.py:340-348*

How kdext_api learns the result

Purely by reading the same Postgres row: kdext_api's own worker runs an adaptive poll loop (`_wait_for_result` — 0.3s for the first 60s, then 2s, bounded by `WORKER_RESULT_TIMEOUT` = 890s) immediately after the hand-off, and `GET /v1/requests/{request_id}` lets external clients poll the identical row directly. *processor.py:487-513*

Retry & dead-letter policy

MECHANISM	TRIGGER	BEHAVIOR
Retry sweep (<code>retry_requeue.py</code>)	Row is <code>TIMEOUT</code> , <code>retry_count < max_retries</code> , and enough backoff time has elapsed	Polls every 60s; backoff = <code>5 minutes × 2^retry_count</code> (5m, 10m, 20m...). Atomically re-claims the row, re-publishes <code>request_id</code> onto the same main queue.
Dead-letter queue	Malformed message only (JSON parse / missing <code>request_id</code>)	Sent to <code>document_processing_dlq</code> , then acked.
Exhausted retries	<code>retry_count >= max_retries</code>	Row simply stops appearing in retry candidates and is left permanently in <code>TIMEOUT</code> — no DLQ message is ever produced for this case , despite the module docstring implying otherwise.
<code>FAILED</code> rows	—	Deliberately excluded from the retry-candidate query — in practice only <code>TIMEOUT</code> rows are ever swept.

SECTION 7

Notifications — Webhooks (webhooks . py)

A durable, signed, retried HTTP push to a URL the customer registers once per API-key tier. Prefix: `/v1/notifications` . Auth: `get_current_external_user` (JWT or API key).

7.1 — Registration & management endpoints

ROUTE	SUCCESS	FAILURE MODES
POST <code>/v1/notifications/register</code>	201 new / 200 updated — returns <code>{webhook_id, key_environment, url, events, is_active, secret, message}</code> . Secret returned in full only here .	400 invalid JSON body · 422 schema validation (bad URL / unsupported event name) <i>webhooks.py:54-134</i>
GET <code>/v1/notifications</code>	200 — secret masked as <code>*****</code> <code><last4></code>	404 no webhook registered for this key's tier <i>webhooks.py:137-164</i>
GET <code>/v1/notifications/all</code>	200 — every tier's registration for the account, secrets masked	— <i>webhooks.py:167-191</i>
DELETE <code>/v1/notifications</code>	200 deactivated (<code>is_active=false</code>)	404 none registered <i>webhooks.py:257-276</i>
POST <code>/v1/notifications/echo</code> (<i>unauthenticated test receiver</i>)	200 <code>{received: true}</code>	400 missing <code>X-Webhook-Id</code> header <i>webhooks.py:194-231</i>
GET <code>/v1/notifications/echo</code>	200 <code>{webhook_id, deliveries[]}</code> (last 20)	404 none registered <i>webhooks.py:234-254</i>

`key_environment` (`production` / `sandbox` / `testing`) is always **server-derived** from whichever API key authenticated the call — a sandbox key can never overwrite the production webhook URL. One row per (`user_id`, `key_environment`) . Updating an existing registration preserves its secret.

7.2 — Trigger points (who calls `publish_document_event` , and when)

LOCATION	EVENT	CIRCUMSTANCE
<code>processor.py:671-692</code>	<code>document.processing.completed</code> / <code>.failed</code>	Duplicate-dispatch race resolved via poll — ensures the webhook still fires even though the fast-path "already processing" reply skips it
<code>processor.py:908-929</code>	same two	Trigger call to <code>kdext_document_api_v2</code> failed, but the row already reached a terminal state via <code>kdext_api</code> 's own poller
<code>processor.py:1364-1395</code>	same two	The main path — normal end-of-job completion or failure
<code>processor.py:1419-1445</code>	<code>document.processing.failed</code>	Unhandled exception anywhere in the job pipeline (top-level catch)

`admin.py:297-323``document.corrected`Admin/validator submits a HITL correction via `PUT /admin/requests/{id}/correction`

Publishing itself is synchronous and `await` ed at the end of the triggering action, but is wrapped so a publish failure can never break document processing — `publish_document_event()` swallows all exceptions and returns `None` on failure. *events.py:142-174*

7.3 — Payload & signing

Delivery headers (every attempt)

```
Content-Type: application/json
X-Event: document.processing.completed
X-Timestamp: 2026-08-07T12:00:00Z
X-Signature: <hex hmac-sha256>
X-Webhook-Id: 42
X-Delivery-Id: <event_id uuid>
```

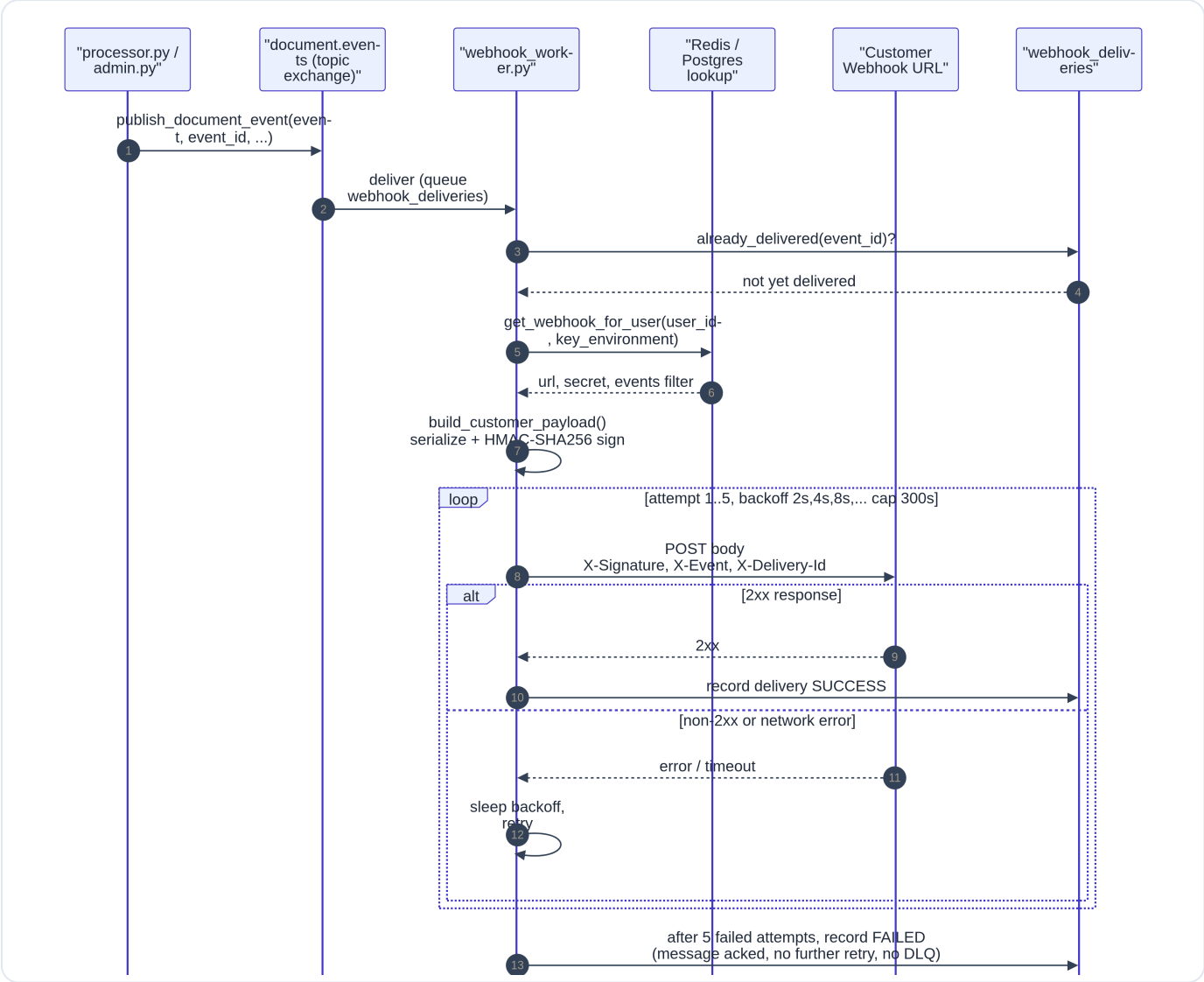
Delivery body (customer-facing fields only)

```
{
  "event": "document.processing.completed",
  "documentId": "req...",
  "documentType": "ReceiptImage",
  "clientDocumentType": "expense",
  "status": "completed",
  "version": 1,
  "timestamp": "2026-08-07T12:00:00Z",
  "error": null
}
```

`signature = HMAC_SHA256(secret, json.dumps(payload, separators=(",", ":"), sort_keys=True))` — canonical, sorted-key JSON is both what's signed and what's sent, so the customer can verify with the same serialization. *webhook_service.py:170-178*

7.4 — Delivery worker & retry/backoff

FIGURE 5 — WEBHOOK DELIVERY WITH RETRY



PARAMETER	DEFAULT
WEBHOOK_PREFETCH	20
WEBHOOK_MAX_RETRIES	5 attempts
WEBHOOK_RETRY_BASE_DELAY / WEBHOOK_RETRY_MAX_DELAY	2.0s / 300.0s — delays are 2s, 4s, 8s, 16s... capped at 300s
WEBHOOK_TIMEOUT_SECONDS	10.0s per attempt

Idempotency: a `WebhookDelivery` row keyed by `event_id` (unique) records `PENDING` → `SUCCESS` | `FAILED`. A redelivered AMQP message whose `event_id` already has status `SUCCESS` is skipped entirely. If no active webhook exists for the tier, or the event isn't in the webhook's `events` subscription filter, the message is simply dropped — no delivery, no retry, no error recorded.

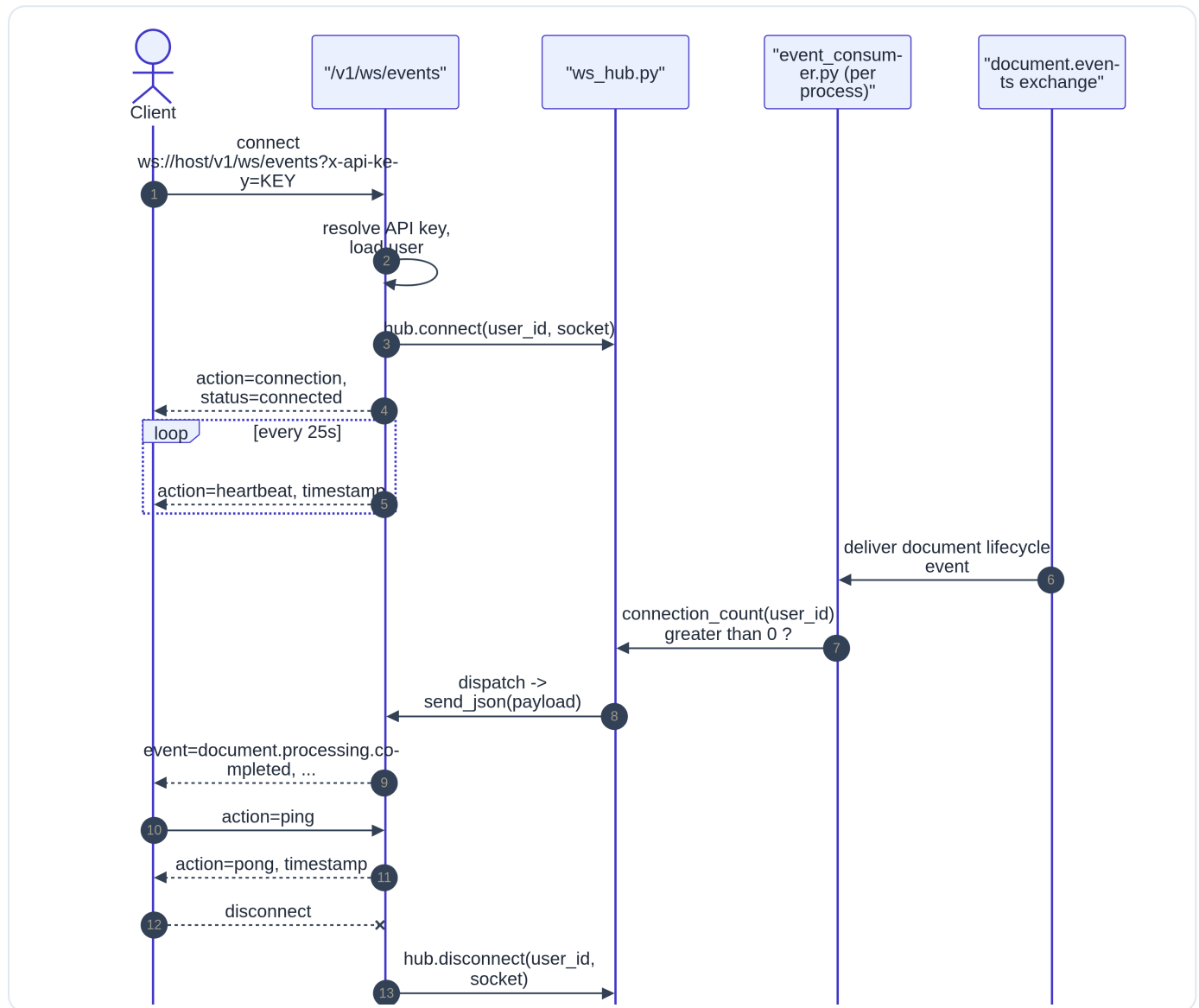
GOTCHA — NO TRUE DEAD-LETTER FOR WEBHOOKS

After 5 failed attempts, the delivery is marked `FAILED` in `webhook_deliveries` and the RabbitMQ message is **still acked** (`requeue=False`). There is no dead-letter queue in `webhook_worker.py` — a permanently-down customer endpoint means that single event is gone for good once the 5-attempt window (up to ~5.5 minutes of backoff) elapses. Recovery relies on the customer polling `GET /v1/notifications/echo` or the underlying `GET /v1/requests/{id}` , not on webhook redelivery.

Notifications — WebSocket `/v1/ws/events`

`events_ws.py` — the live-connection equivalent of a webhook: same three lifecycle events, pushed over an open socket instead of an HTTP callback. "The live connection IS the subscription — no URL to host, no HMAC to verify."

FIGURE 6 — `EVENTS_WS.PY` CONNECTION LIFECYCLE



Connect & auth

Example: `ws://host/v1/ws/events?x-api-key=YOUR_KEY`. Auth happens *before* `.accept()`, via `get_current_external_user_from_websocket`, which resolves the API key from — in order — the `X-API-Key` header, or the `x-api-key` / `X-API-Key` / `api_key` query params. A Redis auth cache is checked first; on miss it falls back to a Postgres lookup against `api_key` / `sandbox_api_key` / `test_api_key`. *dependencies.py:294-370*

Message shapes

DIRECTION	SHAPE	WHEN
Server → Client	<code>{"action": "connection", "status": "connected"}</code>	Immediately after accept
Server → Client	<code>{"action": "heartbeat", "timestamp": <float>}</code>	Every 25s (<code>WS_HEARTBEAT_INTERVAL</code>) — also surfaces a dead peer for pruning
Client → Server	<code>{"action": "ping"}</code>	Optional client keepalive
Server → Client	<code>{"action": "pong", "timestamp": <float>}</code>	Reply to a client ping
Server → Client	<code>{"event", "documentId", "documentType", "clientDocumentType", "status", "version", "timestamp"}</code>	A lifecycle event fires for this user — note: no <code>error</code> field , unlike the webhook payload

The hub mechanism — how one event reaches one socket

- `ws_hub.py`'s `WebSocketHub` is a process-wide singleton: `{user_id: Set[WebSocket]}`, guarded by an `asyncio.Lock`. *ws_hub.py:19-22, 75*
- Because a customer's connection lives on exactly one uvicorn worker process, **every** API process runs its own `event_consumer.py` — an exclusive, auto-delete queue bound to all three routing keys on the `document.events` exchange, so every process sees every event (fan-out). *event_consumer.py:30-55*
- On receipt, if `hub.connection_count(user_id) == 0` in *that* process, it's a no-op — "workers without a matching connection simply drop the event." Otherwise it builds the trimmed customer payload and calls `hub.dispatch(user_id, payload)`, which `send_json()`s to every live socket for that user. *event_consumer.py:63-88; ws_hub.py:47-71*

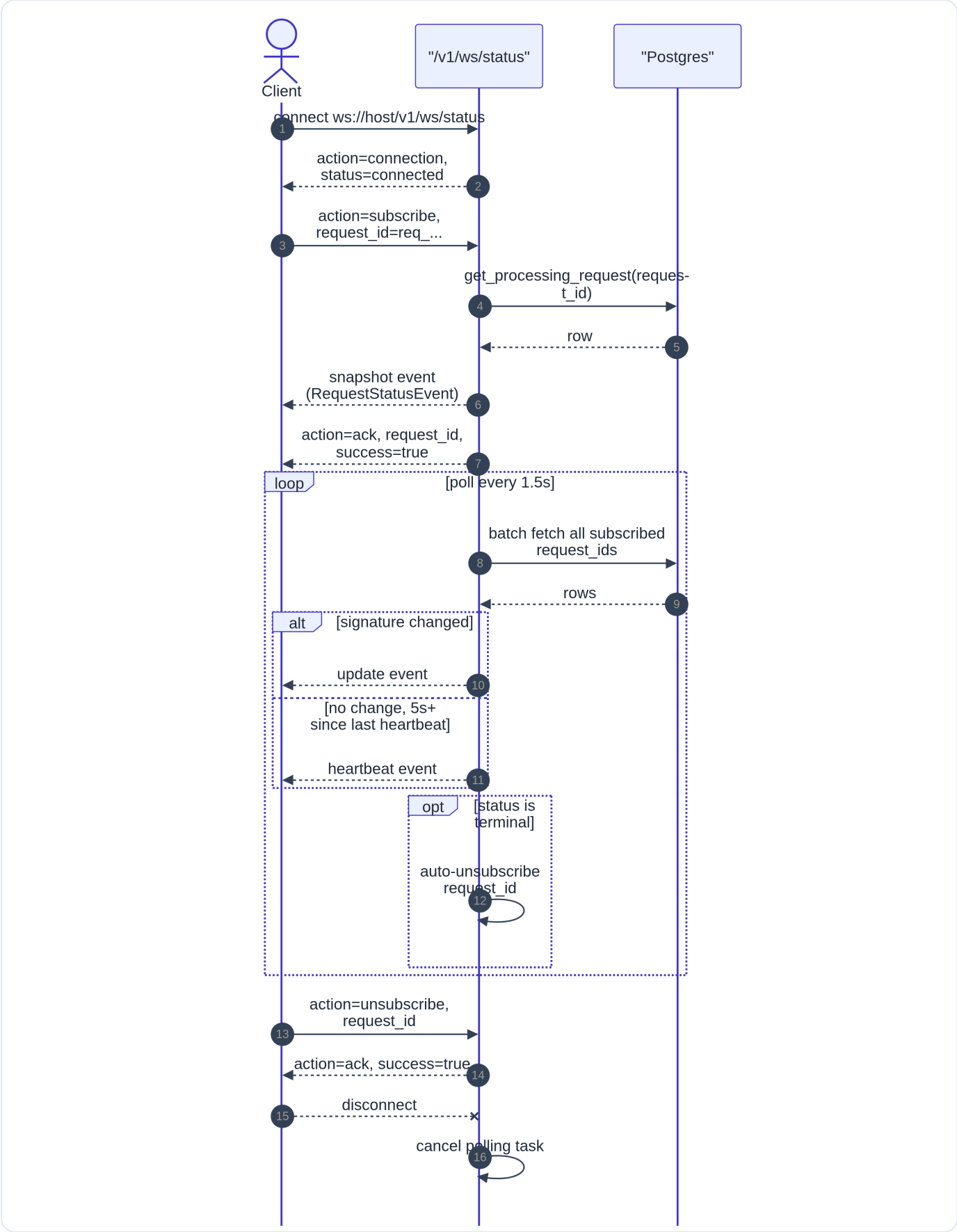
No explicit subscribe/filter semantics exist on this endpoint — a connected socket receives *all* of that user's lifecycle events. Per-event-name filtering is only available through a webhook's `events` list.

SECTION 9

Companion WebSocket — `/v1/ws/status`

`status_ws.py` solves a different problem than `events_ws.py`: instead of "push me everything for my account," a client explicitly subscribes to specific `request_id`s and gets DB-polled status snapshots for exactly those requests. It touches neither RabbitMQ nor `ws_hub` — it polls Postgres directly.

FIGURE 7 — STATUS_WS.PY SUBSCRIPTION & POLLING LIFECYCLE



Client-driven actions

ACTION SENT BY CLIENT	SERVER RESPONSE
<code>{"action": "subscribe", "request_id": "..."} </code>	An immediate snapshot event for that request, then <code>{"action": "ack", "request_id", "success": true} </code> — or <code>success: false</code> with an error string if the request doesn't exist
<code>{"action": "unsubscribe", "request_id": "..."} </code>	<code>{"action": "ack", "request_id", "success": true} </code>
<code>{"action": "ping"} </code>	<code>{"action": "pong", "timestamp"} </code>
anything else	<code>{"action": "error", "error": "Unknown action: ..."} </code>

The background poll loop (every 1.5s, per connection)

All currently-subscribed `request_id` s are fetched in a single batch query per tick — not one query per request. For each row: if a computed content signature (hash of status/timestamps/result/error) changed since last tick, an **update** event fires; else if ≥ 5 s have passed since the last heartbeat, a **heartbeat** event fires (shaped as a full status event, not the lightweight ping/pong of `events_ws.py`). Once a request reaches a terminal status, it is auto-unsubscribed — no more updates for it.

Event schema (`RequestStatusEvent`)

```
{
  "event": "update" | "snapshot" | "heartbeat",
  "request_id": "req...", "document_type": "...", "client_document_type": "...",
  "status": "PROCESSING", "message": "Your document is being processed.",
  "submitted_at": "...", "completed_at": null, "processing_time_ms": null,
  "result_data": null, "formatted_result": null, "error_message": null,
  "is_terminal": false
}
```

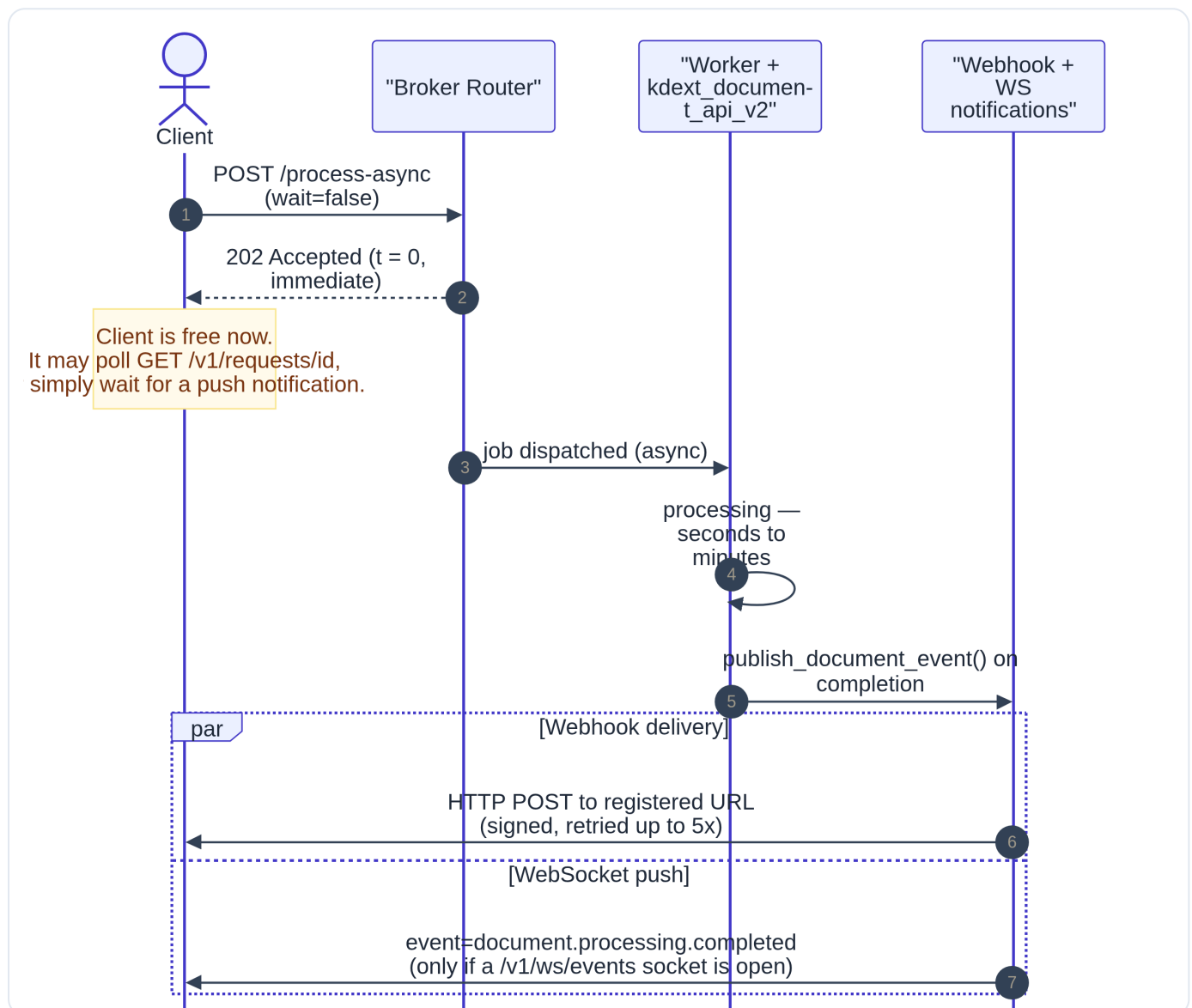
events_ws.py vs. status_ws.py, side by side

	VIA EVENT	VIA STATUS
Data source	RabbitMQ <code>document.events</code> → <code>event_consumer.py</code> → hub push	Direct Postgres polling, every 1.5s
Subscription model	Implicit — all lifecycle events for this user	Explicit — per <code>request_id</code> , subscribe/unsubscribe
Registry	Global singleton, keyed by <code>user_id</code>	Local to the connection, keyed by <code>request_id</code>
Push trigger	Event-driven (AMQP message arrival)	Time-driven (poll diff every 1.5s)
Terminal handling	N/A — events are one-shot	Auto-unsubscribes once terminal

End-to-End Timeline

The single fact that ties every prior section together: **the HTTP response and the two notification channels are on entirely independent clocks.** Whether you used `wait=true` or `wait=false` only changes when the *HTTP call* returns — it never changes when the webhook fires or when the WS push happens, because both are driven off the same `publish_document_event()` call at the end of the job, regardless of how the job was submitted.

FIGURE 8 — COMBINED TIMELINE (WAIT=FALSE SHOWN; WAIT=TRUE ONLY MOVES THE FIRST RESPONSE)



DESIGN CONSEQUENCE

A client that only cares about the eventual result should prefer `wait=false` + a registered webhook (or an open `/v1/ws/events` socket) over holding an HTTP connection open with `wait=true` for up to 900 seconds. `wait=true` is best reserved for low-latency documents where the caller genuinely cannot proceed without an inline result.

SECTION 11

Known Behavioral Quirks

Things the code actually does that diverge from what a docstring, a schema, or a first read would suggest. Worth knowing before building a client or debugging a support ticket.

1 — STALE DOCSTRING ON 422

`broker.py`'s own module docstring claims "A 422 (`not_processed`) is returned up front" for unrecognized documents. The current code never does this from the router — classification moved into the worker, and an unrecognized document now surfaces as DB status `INVALID` (discovered later, via polling/webhook/WS), not as an inline 422.

2 — HTTP 200 CAN MEAN FAILURE (WAIT=TRUE)

The `wait=true` success branch always returns HTTP 200, even when the embedded `status` field is `"failed"` or `"invalid"`. Status-code-only error handling on the client will misclassify these as successes.

3 — DEEP EXCEPTIONS LOSE THEIR STATUS CODE

`save_uploaded_file`'s `HTTPException(400)` / `HTTPException(413)` aren't specially caught in `broker.py` — they fall into the generic `except Exception` and come back as a flat 500.

4 — TWO DIFFERENT STATUS ENUMS, ONE MISSING A VALUE

The Pydantic `JobStatus` enum in `app/schemas/document.py` (lowercase: `pending/processing/completed/failed/cancelled/timeout/invalid`) is **never actually applied** to any response field, and it's missing `queued` — which *is* a real, observable wire value, since `get_request_status` just lowercases whatever the DB column holds. See the reconciliation table in the Appendix.

5 — RESPONSE_MODEL IS COSMETIC

Neither `/process-async` nor `GET /v1/requests/{id}` actually returns a validated Pydantic model instance — both hand-build a dict and wrap it in a raw `Response(...)`. The declared `response_model` on the GET route affects OpenAPI docs only; it is not enforced against the real payload.

6 — RETRY-EXHAUSTION DLQ DOESN'T EXIST

`rabbitmq_manager.py`'s docstring in `kdex_document_api_v2` describes the dead-letter queue as covering both malformed messages *and* exhausted retries. Only the malformed-message path is actually implemented; a row that exhausts `max_retries` simply stops being swept and stays in `TIMEOUT` forever, with no DLQ message produced.

7 — WEBHOOK FAILURE HAS NO DEAD-LETTER EITHER

After 5 failed delivery attempts, `webhook_worker.py` acks the message and records `FAILED` — there is no requeue, no DLQ, and no further automatic retry beyond that one backoff cycle.

8 — TOON, NOT JSON, ON THE HOT PATH

Every response from `broker.py` and the polling route is `media_type="text/plain"` with a TOON-encoded body, not standard `application/json` — clients must decode accordingly. Middleware-layer errors (401/403/429) are, by contrast, real `JSONResponse`.

Appendix

12.1 — Config & timeout reference (kdext_api / app/core/config.py)

SETTING	DEFAULT	ROLE
RABBITMQ_RPC_TIMEOUT	900s	wait=true blocking bound
RABBITMQ_CONNECT_TIMEOUT_SECONDS	15.0s	RPC client connect/channel setup bound
RABBITMQ_PUBLISH_MAX_RETRIES / RETRY_DELAY	3 / 0.5s (linear × attempt)	Publish retry, both publish() and call()
RABBITMQ_REQUEST_QUEUE	document_processing	kdext_api's internal RPC queue name
WORKER_RESULT_TIMEOUT	890s	Worker's own DB-poll ceiling — fires slightly before the 900s RPC timeout
WORKER_RESULT_POLL_INTERVAL_FAST / _WINDOW / _INTERVAL	0.3s / 60s / 2.0s	Worker's adaptive DB poll cadence
WORKER_DRAIN_TIMEOUT_SECONDS	960s	Graceful-shutdown drain window
MAX_FILE_SIZE	500 MB	Pre-S3 size guard (413)
Broker poll hint (constant, not a setting)	5s	poll_interval_seconds / Retry-After on every 202/503
RABBITMQ_EVENTS_EXCHANGE	document.events	Notification fan-out exchange (topic)
WEBHOOK_DELIVERY_QUEUE	webhook_deliveries	Durable webhook delivery queue
WEBHOOK_PREFETCH / MAX_RETRIES	20 / 5	Webhook worker concurrency & retry cap
WEBHOOK_RETRY_BASE_DELAY / _MAX_DELAY	2.0s / 300.0s	Exponential backoff, capped
WEBHOOK_TIMEOUT_SECONDS	10.0s	Per-attempt HTTP timeout
WS_HEARTBEAT_INTERVAL (events_ws)	25.0s	Server heartbeat cadence
POLL_INTERVAL_SECONDS (status_ws)	1.5s	Background DB poll cadence
HEARTBEAT_INTERVAL_SECONDS (status_ws)	5.0s	Per-subscription heartbeat-if-unchanged threshold

12.2 — Status enum reconciliation

SURFACE	VALUES
Pydantic JobStatus document.py:10-18 (declared, effectively unused)	pending, processing, completed, failed, cancelled, timeout, invalid
Actual kdext_api DB column document_processing_request.status (ground truth, wire value is lowercased)	PENDING, QUEUED, PROCESSING, COMPLETED, FAILED, CANCELLED, TIMEOUT, INVALID — note QUEUED, absent from JobStatus

kdext_document_api_v2 `ProcessingStatus` enum
job_models.py:21-34

`PENDING, QUEUED, PROCESSING, COMPLETED, FAILED, CANCELLED, TIMEOUT, INVALID` — matches the real DB column; `CANCELLED` has no writer in either repo, reserved for upstream use

12.3 — Full endpoint index (this flow only)

METHOD & PATH	FILE	PURPOSE
<code>POST /v1/documents/process-async</code>	<code>routers/broker.py</code>	Submit a document, sync or async
<code>GET /v1/requests/{request_id}</code>	<code>routers/requests.py</code>	Poll for status/result (also shadows a dead route of the same path in <code>status.py</code>)
<code>POST /v1/notifications/register</code>	<code>routers/webhooks.py</code>	Register/update a webhook URL for the caller's key tier
<code>GET /v1/notifications / /all</code>	<code>routers/webhooks.py</code>	Read current registration(s)
<code>DELETE /v1/notifications</code>	<code>routers/webhooks.py</code>	Deactivate a webhook
<code>POST / GET /v1/notifications/echo</code>	<code>routers/webhooks.py</code>	Unauthenticated test receiver + delivery history
<code>WS /v1/ws/events</code>	<code>routers/events_ws.py</code>	Live push of the same 3 lifecycle events as the webhook
<code>WS /v1/ws/status</code>	<code>routers/status_ws.py</code>	Explicit per-request-id polling stream

kdext_api & kdext_document_api_v2 — process-async & notification pipeline reference · generated from source, file:line cited throughout